

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

Дисципліна
« Прикладна математика »

Лабораторна робота № 1

Виконав:	Дацишин Володимир	Перевірила:	Ковтун Оксана Іванівна
Група	ІПЗ-23		
Форма навчання	денна	Оцінка	
Спеціальність	121		
2026			

Тема: Застосування лінійної та векторної алгебри з використанням бібліотек мови програмування Python

Варіант: 8

Завдання №1

1. Знайдіть $f(A)$, якщо:

$$A = \begin{pmatrix} 2 & 1 & 3 \\ -3 & 0 & -1 \\ 4 & 2 & -1 \end{pmatrix}, f(x) = -2x^2 + 8x - 6.$$

Основні теоретичні відомості:

Щоб знайти значення функції $f(x)$ від матриці A , необхідно замінити змінну x на матрицю A . При цьому звичайні числа (вільні члени многочлена) множаться на одиничну матрицю I того ж розміру, що й матриця A . Тобто рівняння набуде вигляду: $f(A) = -2 \cdot A^2 + 8 \cdot A - 6 \cdot I$.

Операції над матрицями:

- **Множення матриці на число:** кожен елемент матриці множиться на це число.
- **Піднесення матриці до степеня (A^2):** матриця множиться сама на себе ($A \cdot A$). Множення відбувається за правилом "рядок на стовпець" (можливо лише для квадратних матриць).
- **Додавання/віднімання матриць:** виконується поелементно для матриць однакового розміру.
- **Одинична матриця (I):** квадратна матриця, у якій на головній діагоналі стоять одиниці, а всі інші елементи - нулі.

Хід рішення:

Цей код обчислює значення матричного многочлена $f(A) = -2A^2 + 8A - 6I$. Спочатку програма задає початкову матрицю A та створює одиничну матрицю того ж розміру, яка є обов'язковою, оскільки за правилами алгебри вільний член (число 6) має множитися саме на неї. Потім матриця A підноситься до квадрату за допомогою вбудованої функції `matrix_power`. Після цього виконується фінальний розрахунок: отриманий квадрат матриці множиться на -2, початкова матриця множиться на 8, а одинична — на 6, після чого всі три матриці поелементно додаються, формуючи кінцевий результат для виводу на екран

Код

```
def task1(): 2 usages
    A = np.array(
        [
            [2, 1, 3],
            [-3, 0, -1],
            [4, 2, -1]
        ]
    )

    # Створення квадратної матриці 3x3
    # .eye() створює матрицю типу float, 'dtype=int' - явно вказує на цільний тип чисел
    I = np.eye(N=3, dtype=int)

    f_A = -2 * np.linalg.matrix_power(A, n=2) + 8 * A - 6 * I
    print("\n=== Task 1 ===")
    print(f"Result:\n{f_A}")
```

Результат

```
=== Task 1 ===
Result:
[[-16  -8  20]
 [ -4   4   8]
 [ 36  12 -36]]
```

Перевірка

$$A = \begin{pmatrix} 2 & 1 & 3 \\ -3 & 0 & -1 \\ 4 & 2 & -1 \end{pmatrix}, f(x) = -2x^2 + 8x - 6$$
$$f(A) = -2A^2 + 8A - 6I$$
$$A^2 = \begin{pmatrix} 2 & 1 & 3 \\ -3 & 0 & -1 \\ 4 & 2 & -1 \end{pmatrix} \cdot \begin{pmatrix} 2 & 1 & 3 \\ -3 & 0 & -1 \\ 4 & 2 & -1 \end{pmatrix} =$$
$$= \begin{pmatrix} 2 \cdot 2 + 1 \cdot (-3) + 3 \cdot 4 & 2 \cdot 1 + 1 \cdot 0 + 3 \cdot 2 & 2 \cdot 3 + 1 \cdot (-1) + 3 \cdot (-1) \\ -3 \cdot 2 + 0 \cdot (-3) + (-1) \cdot 4 & -3 \cdot 1 + 0 \cdot 0 + (-1) \cdot 2 & -3 \cdot 3 + 0 \cdot (-1) + (-1) \cdot (-1) \\ 4 \cdot 2 + 2 \cdot (-3) + (-1) \cdot 4 & 4 \cdot 1 + 2 \cdot 0 + (-1) \cdot 2 & 4 \cdot 3 + 2 \cdot (-1) + (-1) \cdot (-1) \end{pmatrix} =$$
$$= \begin{pmatrix} 13 & 8 & 2 \\ -10 & -5 & -8 \\ -2 & 2 & 11 \end{pmatrix}$$
$$-2A^2 = -2 \cdot \begin{pmatrix} 13 & 8 & 2 \\ -10 & -5 & -8 \\ -2 & 2 & 11 \end{pmatrix} = \begin{pmatrix} -26 & -16 & -4 \\ 20 & 10 & 16 \\ 4 & -4 & -22 \end{pmatrix}$$
$$8A = 8 \cdot \begin{pmatrix} 2 & 1 & 3 \\ -3 & 0 & -1 \\ 4 & 2 & -1 \end{pmatrix} = \begin{pmatrix} 16 & 8 & 24 \\ -24 & 0 & -8 \\ 32 & 16 & -8 \end{pmatrix}$$
$$-6I = \begin{pmatrix} -6 & 0 & 0 \\ 0 & -6 & 0 \\ 0 & 0 & -6 \end{pmatrix}$$
$$f(A) = \begin{pmatrix} -26 & -16 & -4 \\ 20 & 10 & 16 \\ 4 & -4 & -22 \end{pmatrix} + \begin{pmatrix} 16 & 8 & 24 \\ -24 & 0 & -8 \\ 32 & 16 & -8 \end{pmatrix} + \begin{pmatrix} -6 & 0 & 0 \\ 0 & -6 & 0 \\ 0 & 0 & -6 \end{pmatrix} =$$
$$= \begin{pmatrix} -26+16-6 & -16+8 & -4+24 \\ 20-24 & 10-6 & 16-8 \\ 4+32 & -4+16 & -22-8-6 \end{pmatrix} = \begin{pmatrix} -16 & -8 & 20 \\ -4 & 4 & 8 \\ 36 & 12 & -36 \end{pmatrix}$$

Завдання №2:

2. Розв'яжіть матричне рівняння.

$$X \cdot A = B, A = \begin{pmatrix} 4 & 3 & 6 \\ 5 & 4 & 7 \\ 6 & 5 & 7 \end{pmatrix}, B = \begin{pmatrix} 2 & 1 & 5 \\ 0 & 0 & -1 \end{pmatrix}.$$

Основні теоретичні відомості

- **Обернена матриця (A^{-1}):** Це така матриця, при множенні на яку початкової матриці утворюється одинична матриця ($A \cdot A^{-1} = A^{-1} \cdot A = E$). Вона існує лише для квадратних матриць, визначник (детермінант) яких не дорівнює нулю ($\det A \neq 0$).
- **Розв'язання матричних рівнянь:** Для рівняння виду $X \cdot A = B$, щоб "звільнити" невідому матрицю X , необхідно обидві частини рівняння домножити на обернену матрицю A^{-1} справа (оскільки множення матриць не є комутативним, тобто $A \cdot B \neq B \cdot A$): $X \cdot A \cdot A^{-1} = B \cdot A^{-1}$. Оскільки $A \cdot A^{-1} = I$, а $X \cdot I = X$, отримуємо формулу розв'язку: $X = B \cdot A^{-1}$.

Хід рішення:

Цей код розв'язує матричне рівняння виду $X \cdot A = B$, де для знаходження невідомої матриці X за правилами алгебри необхідно помножити матрицю B на обернену матрицю A^{-1} справа (тобто $X = B \cdot A^{-1}$). Спочатку програма ініціалізує задані матриці A та B як двовимірні масиви NumPy. Потім за допомогою функції `np.linalg.inv(A)` обчислюється обернена матриця до A , після чого виконується матричне множення матриці B на цю обернену матрицю за допомогою оператора `@`. Для усунення дрібних похибок машинної арифметики з плаваючою комою отриманий результат округлюється та приводиться до цілочисельного типу, після чого знайдена кінцева матриця X виводиться на екран.

Код

```
def task2(): 2 usages
    A = np.array(
        [
            [4, 3, 6],
            [5, 4, 7],
            [6, 5, 7]
        ]
    )
    B = np.array(
        [
            [2, 1, 5],
            [0, 0, -1]
        ]
    )

    # .linalg.inv() = A^-1, іншими словами обернена матриця A,
    # символ '@' використовується для множення матриць
    x = B @ (np.linalg.inv(A))
    x = np.round(x).astype(int)

    print("\n=== Task 2 ===")
    print(f"Result:\n{x}")
```

Результат

```
=== Task 2 ===
Result:
[[ 2  0 -1]
 [ 1 -2  1]]
```

Перевірка

$$x \cdot A = B, A = \begin{pmatrix} 4 & 3 & 6 \\ 5 & 4 & 7 \\ 6 & 5 & 7 \end{pmatrix}, B = \begin{pmatrix} 2 & 1 & 5 \\ 0 & 0 & -1 \end{pmatrix}$$
$$x \cdot A \cdot A^{-1} = B \cdot A^{-1} \Rightarrow x = B A^{-1}$$
$$\Delta = 4 \cdot \begin{vmatrix} 4 & 7 \\ 5 & 7 \end{vmatrix} - 3 \cdot \begin{vmatrix} 5 & 7 \\ 6 & 7 \end{vmatrix} + 6 \cdot \begin{vmatrix} 5 & 4 \\ 6 & 5 \end{vmatrix} =$$
$$= 4 \cdot (28 - 35) - 3 \cdot (35 - 42) + 6 \cdot (25 - 24) =$$
$$= -28 + 21 + 6 = -1 \neq 0, \text{ отже } A^{-1} \text{ існує}$$
$$\begin{aligned} A_{11} &= 28 - 35 = -7 \\ A_{12} &= -(35 - 42) = 7 \\ A_{13} &= 25 - 24 = 1 \\ A_{21} &= -(21 - 30) = 9 \\ A_{22} &= 28 - 36 = -8 \\ A_{23} &= -(20 - 18) = -2 \\ A_{31} &= 21 - 24 = -3 \\ A_{32} &= -(28 - 30) = 2 \\ A_{33} &= 40 - 15 = 1 \end{aligned}$$
$$A^{-1} = \frac{1}{-1} \begin{pmatrix} -7 & 9 & +3 \\ 7 & -8 & 2 \\ 1 & -2 & 1 \end{pmatrix} = \begin{pmatrix} 7 & -9 & -3 \\ -7 & 8 & -2 \\ -1 & 2 & -1 \end{pmatrix}$$
$$x = B \cdot A^{-1} = \begin{pmatrix} 2 & 1 & 5 \\ 0 & 0 & -1 \end{pmatrix} \cdot \begin{pmatrix} 7 & -9 & -3 \\ -7 & 8 & -2 \\ -1 & 2 & -1 \end{pmatrix} =$$
$$= \begin{pmatrix} 2 \cdot 7 + 1 \cdot (-7) + 5 \cdot (-1) & -18 + 8 + 10 & 6 - 2 - 5 \\ 0 + 0 + 1 & 0 + 0 - 2 & 0 + 0 - 1 \end{pmatrix} =$$
$$= \begin{pmatrix} 2 & 0 & -1 \\ 1 & -2 & 1 \end{pmatrix}$$

Завдання №3:

3. Дослідіть СЛАР, задану розширеною матрицею, на сумісність і в разі сумісності знайдіть її загальний розв'язок.

$$\left(\begin{array}{cccc|c} 1 & 2 & 1 & 0 & 8 \\ 0 & 1 & 3 & 1 & 15 \\ 4 & 0 & 1 & 1 & 11 \\ 1 & 1 & 0 & 5 & 23 \end{array}\right).$$

Основні теоретичні відомості

- **Ранг матриці (r або rank):** Максимальна кількість лінійно незалежних рядків (або стовпців) матриці.
- **Теорема Кронекера-Капеллі (дослідження на сумісність):** Система лінійних алгебраїчних рівнянь (СЛАР) має розв'язок (є сумісною) тоді і тільки тоді, коли ранг основної матриці системи A дорівнює рангу розширеної матриці A^+ (матриці, до якої додано стовпець вільних членів).
 - Якщо $\text{rank}(A) \neq \text{rank}(A^+)$ — система розв'язків не має.
 - Якщо $\text{rank}(A) = \text{rank}(A^+) = n$ (кількість невідомих) — система має єдиний розв'язок.
 - Якщо $\text{rank}(A) = \text{rank}(A^+) < n$ — система має безліч розв'язків.
- **Розв'язання СЛАР:** Якщо система сумісна і визначена (має єдиний розв'язок), її можна розв'язати методом Крамера, матричним методом ($x = A^{-1} \cdot b$) або методом Гаусса.

Хід рішення:

Використано наступні функції бібліотеки **numpy as np**:

np.column_stack((A, b)) - функція, яка приєднує вектор вільних членів b до основної матриці коефіцієнтів A у вигляді додаткового стовпця, утворюючи таким чином розширену матрицю системи.

np.linalg.matrix_rank(...) - обчислює ранг матриці. Ця функція застосовується двічі: для знаходження рангу основної матриці та рангу розширеної матриці. Порівнявши ці дві величини, ми досліджуємо систему на сумісність згідно з теоремою Кронекера-Капеллі (якщо ранги рівні, система сумісна і має розв'язок).

np.linalg.solve(A, b) - функція, яка безпосередньо розв'язує систему лінійних алгебраїчних рівнянь, обчислюючи і повертаючи вектор невідомих значень (загальний розв'язок). Вона викликається лише за умови, що попередня перевірка підтвердила рівність рангів.

np.round(...).astype(int) - використовується для округлення знайдених коренів системи та перетворення їх на цілочисельний тип, щоб позбутися можливих дрібних похибок машинної арифметики під час виводу результату.

Код

```
def task3(): 2 usages
    A = np.array(
        object: [
            [1, 2, 1, 0],
            [0, 1, 3, 1],
            [4, 0, 1, 1],
            [1, 1, 0, 5]
        ],
        dtype=float
    )
    b = np.array(object: [8, 15, 11, 23], dtype=float)

    # Розширена матриця
    A_aug = np.column_stack((A, b))

    # Обчислення рангів матриць, якщо рівні - система сумісна і має один розв'язок
    rank_A = np.linalg.matrix_rank(A)
    rank_A_aug = np.linalg.matrix_rank(A_aug)

    print("\n=== Task 3 ===")
    print(f"Rank of A matrix: {rank_A}")
    print(f"Rank of A augmented: {rank_A_aug}")

    # Перевірка сумісності
    if rank_A == rank_A_aug:
        x = np.linalg.solve(A, b)
        x_rounded = np.round(x).astype(int)
        print(
            f"Consistent system. Unique solution: x1={x_rounded[0]}, "
            f"x2={x_rounded[1]}, x3={x_rounded[2]}, x4={x_rounded[3]}"
        )
    else:
        print("System is not consistent (no solution).")
```

Результат

```
=== Task 3 ===
Rank of A matrix: 4
Rank of A augmented: 4
Consistent system. Unique solution: x1=1, x2=2, x3=3, x4=4
```

Перевірка

Handwritten Gaussian elimination steps for a 4x4 system:

$$\begin{pmatrix} 1 & 2 & 1 & 0 & | & 8 \\ 0 & 1 & 3 & 1 & | & 15 \\ 4 & 0 & 1 & 1 & | & 11 \\ 1 & 1 & 0 & 5 & | & 23 \end{pmatrix} \xrightarrow{R_3 - 4R_1, R_4 - R_1} \begin{pmatrix} 1 & 2 & 1 & 0 & | & 8 \\ 0 & 1 & 3 & 1 & | & 15 \\ 0 & -8 & -3 & 1 & | & -21 \\ 0 & -1 & -1 & 5 & | & 15 \end{pmatrix} \xrightarrow{R_3 \leftrightarrow R_4} \begin{pmatrix} 1 & 2 & 1 & 0 & | & 8 \\ 0 & 1 & 3 & 1 & | & 15 \\ 0 & -1 & -1 & 5 & | & 15 \\ 0 & -8 & -3 & 1 & | & -21 \end{pmatrix} \xrightarrow{R_3 \times (-1)} \begin{pmatrix} 1 & 2 & 1 & 0 & | & 8 \\ 0 & 1 & 3 & 1 & | & 15 \\ 0 & 1 & 1 & -5 & | & -15 \\ 0 & -8 & -3 & 1 & | & -21 \end{pmatrix} \xrightarrow{R_3 - R_2} \begin{pmatrix} 1 & 2 & 1 & 0 & | & 8 \\ 0 & 1 & 3 & 1 & | & 15 \\ 0 & 0 & -2 & -6 & | & -30 \\ 0 & -8 & -3 & 1 & | & -21 \end{pmatrix} \xrightarrow{R_3 \div (-2)} \begin{pmatrix} 1 & 2 & 1 & 0 & | & 8 \\ 0 & 1 & 3 & 1 & | & 15 \\ 0 & 0 & 1 & 3 & | & 15 \\ 0 & -8 & -3 & 1 & | & -21 \end{pmatrix} \xrightarrow{R_4 + 8R_3} \begin{pmatrix} 1 & 2 & 1 & 0 & | & 8 \\ 0 & 1 & 3 & 1 & | & 15 \\ 0 & 0 & 1 & 3 & | & 15 \\ 0 & 0 & 21 & 25 & | & 123 \end{pmatrix} \xrightarrow{R_4 \div 21} \begin{pmatrix} 1 & 2 & 1 & 0 & | & 8 \\ 0 & 1 & 3 & 1 & | & 15 \\ 0 & 0 & 1 & 3 & | & 15 \\ 0 & 0 & 1 & 3 & | & 33 \end{pmatrix} \xrightarrow{R_4 - R_3} \begin{pmatrix} 1 & 2 & 1 & 0 & | & 8 \\ 0 & 1 & 3 & 1 & | & 15 \\ 0 & 0 & 1 & 3 & | & 15 \\ 0 & 0 & 0 & 0 & | & -18 \end{pmatrix} \xrightarrow{R_4 \div (-18)} \begin{pmatrix} 1 & 2 & 1 & 0 & | & 8 \\ 0 & 1 & 3 & 1 & | & 15 \\ 0 & 0 & 1 & 3 & | & 15 \\ 0 & 0 & 0 & 1 & | & 4 \end{pmatrix}$$

$\Gamma_A = 4, \Gamma_{\bar{A}} = 4, n = 4, \Gamma(A) = \Gamma(\bar{A})$, отже система є сумісною та має єдиний розв'язок.

$x_4 = 4$

$$\begin{aligned} x_3 + 12 &= 15 \Rightarrow x_3 = 3 \\ x_2 + 9 + 4 &= 15 \Rightarrow x_2 = 2 \\ x_1 + 4 + 3 &= 8 \Rightarrow x_1 = 1 \end{aligned}$$

В.о. $x_1 = 1, x_2 = 2, x_3 = 3, x_4 = 4$

Завдання №4:

4. Обчислить:

а) $(3\vec{a} + 2\vec{b})(-\vec{a} + 3\vec{b})$; б) $|2\vec{a} + 3\vec{b}|$, якщо $|\vec{a}| = 4\sqrt{2}$, $|\vec{b}| = 3$, $\varphi = \frac{\pi}{4}$.

Основні теоретичні відомості

- **Скалярний добуток векторів:** Число, що дорівнює добутку модулів цих векторів на косинус кута між ними
- **Алгебраїчні властивості:** Розкриття дужок у векторних виразах відбувається за правилами звичайної алгебри.
- **Скалярний квадрат:** Скалярний добуток вектора на самого себе дорівнює квадрату його модуля (довжини).
- **Довжина (модуль) вектора:** Може бути знайдена як корінь із його скалярного квадрата. Для обчислення модуля складного виразу (наприклад, пункт б) потрібно спочатку піднести вираз до квадрату, підставити числові значення, а потім добути корінь із результату.

Хід рішення:

Використано наступні функції бібліотек **sympy** (як **sp**) та **numpy** (як **np**):

sp.symbols('a b') - функція, яка ініціалізує символічні змінні для представлення векторів, що дозволяє працювати з ними як з математичними об'єктами алгебри до етапу підстановки конкретних чисел.

sp.expand(...) - функція, яка символічно розкриває дужки у заданому векторному виразі, перемножуючи вектори та зводячи вираз до комбінації їх скалярних квадратів і скалярних добутків.

np.cos(angle) - обчислює косинус кута між векторами. Використовується для знаходження базового скалярного добутку векторів за формулою, що множить їхні довжини на косинус кута між ними.

.subs(...) - метод бібліотеки SymPy, який здійснює підстановку обчислених числових значень (квадратів модулів векторів та їх скалярного добутку) замість символічних змінних у розкритий вираз для отримання фінального результату пункту "а".

np.sqrt(...) - функція для обчислення квадратного кореня. У пункті "б" вона застосовується для знаходження модуля (довжини) результуючого вектора шляхом добування кореня з його розгорнутого скалярного квадрата.

Код

```
def task4(): 2 usages
    a, b = sp.symbols('a b')
    expression = (3 * a + 2 * b) * (-a + 3 * b)
    # Розширення виразу
    expanded_expression = sp.expand(expression)
    # Довжини векторів та кут між ними
    a_mag = 4 * np.sqrt(2)
    b_mag = 3
    angle = np.pi / 4

    # Обчислення скалярного добутку
    scalar_product = a_mag * b_mag * np.cos(angle)

    # Підстановка відповідних значень у вираз
    result = expanded_expression.subs(
        {
            a**2: a_mag**2,
            b**2: b_mag**2,
            a*b: scalar_product
        }
    )

    print("\n=== Task 4(a) ===")
    print(f"Expanded expression: {expanded_expression}")
    print(f"Result after substitution: {result}")

    result = np.sqrt(4 * (a_mag**2) + 12 * scalar_product + 9 * (b_mag**2))
    print("\n=== Task 4(b) ===")
    print(f"Result for |2a + 3b|: {result:.6f}")
```

Результат

```
=== Task 4(a) ===
Expanded expression: -3*a**2 + 7*a*b + 6*b**2
Result after substitution: 42.00000000000000

=== Task 4(b) ===
Result for |2a + 3b|: 18.788294
```

Перевірка

a) $(3\vec{a} + 2\vec{b})(-\vec{a} + 3\vec{b})$

$$\vec{a} \cdot \vec{b} = |\vec{a}| \cdot |\vec{b}| \cdot \cos \varphi = 4\sqrt{2} \cdot 3 \cdot \cos \frac{\pi}{4} =$$
$$= 12\sqrt{2} \cdot \frac{\sqrt{2}}{2} = 12$$
$$|\vec{a}|^2 = |\vec{a}|^2 = (4\sqrt{2})^2 = 32, |\vec{b}|^2 = 9$$
$$(3\vec{a} + 2\vec{b})(-\vec{a} + 3\vec{b}) = -3\vec{a}^2 + 9\vec{a}\vec{b} - 2\vec{b}\vec{a} + 6\vec{b}^2 =$$
$$= -3\vec{a}^2 + 7\vec{a}\vec{b} + 6\vec{b}^2 = -3 \cdot 32 + 7 \cdot 12 + 6 \cdot 9 = 42$$

б) $|2\vec{a} + 3\vec{b}|$

$$|2\vec{a} + 3\vec{b}|^2 = (2\vec{a} + 3\vec{b})^2 = 4\vec{a}^2 + 12\vec{a}\vec{b} + 9\vec{b}^2 =$$
$$= 4 \cdot 32 + 12 \cdot 12 + 9 \cdot 9 = 353$$
$$|2\vec{a} + 3\vec{b}| = \sqrt{353} \approx 18,788294$$

Завдання №5:

5. Обчисліть площу грані ABC і об'єм $ABCD$, вершини якої містяться в точках:

$$A = (0; -3; 5), B = (-3; -1; 1), C = (2; -5; 2), D = (4; 3; 6).$$

Основні теоретичні відомості

- **Вектор за його точками:** Координати вектора AB знаходяться відніманням координат початку (точки A) від координат кінця (точки B)
- **Площа грані (трикутника) та Векторний добуток:** Геометричний зміст модуля векторного добутку - це площа паралелограма, побудованого на цих векторах. Відповідно, площа трикутника ABC дорівнює половині цієї площі
- **Об'єм піраміди (тетраедра) та Мішаний добуток:** Мішаний добуток трьох векторів за модулем дорівнює об'єму паралелепіпеда, побудованого на них. Об'єм тетраедра (трикутної піраміди) становить $1/6$ від цього об'єму. Мішаний добуток можна обчислити як скалярний добуток вектора на результат векторного добутку.

Хід рішення:

Використано наступні функції бібліотеки **numpy as np**:

Спочатку в коді розраховуються координати векторів AB , AC та AD шляхом віднімання координат початкової точки A від координат точок B , C та D відповідно. Далі застосовуються функції:

np.cross(AB , AC) - функція, яка обчислює векторний добуток векторів AB і AC .

np.linalg.norm(...) - обчислює довжину вектора (результату векторного добутку).

Ця величина геометрично дорівнює площі паралелограма, побудованого на векторах AB і AC . Помноживши це значення на 0.5 (або поділивши на 2), програма знаходить площу трикутної грані ABC .

np.dot(cross_prod, AD) - функція, яка обчислює скалярний добуток між вектором AD і результатом векторного добутку AB і AC (знаходить мішаний добуток трьох векторів). Якщо ми використаємо **abs()** (абсолютне значення), то це і буде об'єм паралелепіпеда, побудованого на векторах AB , AC та AD .

Використавши коефіцієнт $1/6$ від цього модуля мішаного добутку, програма обчислює остаточний об'єм трикутної піраміди (тетраедра) $ABCD$.

Код

```
def task5(): 2 usages
    A = np.array([0, -3, 5])
    B = np.array([-3, -1, 1])
    C = np.array([2, -5, 2])
    D = np.array([4, 3, 6])

    AB = B - A
    AC = C - A
    AD = D - A

    cross_prod = np.cross(AB, AC)

    S_abc = 0.5 * np.linalg.norm(cross_prod)

    mixed_prod = np.dot(cross_prod, AD)
    V_abcd = (1 / 6) * abs(mixed_prod)

    print("\n=== Task 5 ===")
    print(f"Area of a face of ABC: {S_abc:.4f}")
    print(f"Volume of a piramid: {V_abcd:.1f}")
```

Результат

```
=== Task 5 ===
Area of a face of ABC: 11.0567
Volume of a piramid: 26.0
```

Перевірка

$$\begin{aligned} & A(0; -3; 5), B(-3; -1; 1), C(2; -5; 2), D(4; 3; 6) \\ & \vec{AB} = (-3; 2; -4) \\ & \vec{AC} = (2; -2; -3) \\ & \vec{AD} = (4; 6; 1) \\ & S_{ABC} = \frac{1}{2} |\vec{AB} \times \vec{AC}| \\ & \vec{AB} \times \vec{AC} = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ -3 & 2 & -4 \\ 2 & -2 & -3 \end{vmatrix} = \vec{i} \cdot \begin{vmatrix} 2 & -4 \\ -2 & -3 \end{vmatrix} - \vec{j} \cdot \begin{vmatrix} -3 & -4 \\ 2 & -3 \end{vmatrix} + \vec{k} \cdot \begin{vmatrix} -3 & 2 \\ 2 & -2 \end{vmatrix} \\ & = -14\vec{i} - 17\vec{j} + 2\vec{k} \\ & |\vec{AB} \times \vec{AC}| = \sqrt{(-14)^2 + (-17)^2 + 2^2} = \sqrt{196 + 289 + 4} = \sqrt{489} \\ & S_{ABC} = \frac{\sqrt{489}}{2} \approx 11,0567 \text{ (кв. од.) } B. \\ & V_{ABCD} = \frac{1}{6} |(\vec{AB} \times \vec{AC}) \cdot \vec{AD}| \\ & (\vec{AB} \times \vec{AC}) \cdot \vec{AD} = -14 \cdot 4 - 17 \cdot 6 + 2 \cdot 1 = -156 \\ & V_{ABCD} = \frac{1}{6} \cdot |-156| = \frac{156}{6} = 26 \text{ (куб. од.) } B. \end{aligned}$$

Висновок: під час виконання лабораторної роботи мною було практично закріплено фундаментальні поняття лінійної та векторної алгебри, такі як матричні обчислення, аналіз СЛАР на сумісність та знаходження геометричних величин за допомогою добутків векторів. Використання бібліотек NumPy та SymPy мовою Python наочно продемонструвало, як сучасні програмні інструменти дозволяють швидко та ефективно автоматизувати складні математичні розрахунки. Це підтверджує важливість поєднання теоретичної математичної бази з навичками програмування для вирішення реальних прикладних та інженерних задач.